A Pilot Evaluation of REA-Assisted Functional Reconstruction

Three experimental arms on 12 JavaScript tasks

Technical Report v1.1 · 15 October 2026 · REA Functional Reconstruction Evaluation · Results [1] / Methods [2]

Abstract

We investigate whether REA improves bounded functional reconstruction under a shared base model and budget caps. Twelve tasks from six fixed-version open-source JavaScript libraries each provide ten examples and 200 hidden cases for evaluating a Python implementation. We compare behavior queries (A), additional direct code inspection (B), and additional required REA analysis and tracing (C), with one generation per task and arm: 36 valid runs. Under frozen protocol v0.2.0, software-macro pass rates are 53.79%, 47.50%, and 78.33%, respectively; C−B is +30.83 percentage points. A validator false positive affected P12-C. An unchanged-code replay raises the sensitivity result for C to 86.67%, reported separately. These observations motivate further evaluation, but shared templates, single generations, execution failures, and workflow-guidance confounding prevent claims of stable or independently attributable REA gains.

Keywords: functional reconstruction; reverse-engineering agents; REA; behavioral equivalence; pilot evaluation

1. Research questions and scope

REA offers reverse-engineering tools for applications, JavaScript, and native binaries [3]. We examine only a bounded setting: reconstructing Python behavior from library interfaces and minified JavaScript artifacts. This is neither an evaluation of the full REA system nor a reproduction of its authors' experiments. We ask how C changes pass rates relative to B, what REA evidence is actually used, and which failures and resource costs accompany the differences.

2. Meaning of the experimental arms

Arm	Available information and required steps	Purpose
A: behavior queries	Task contract, library name/version, ten examples, and queries on chosen inputs. No target-code access or REA.	Reconstruction from behavior and model prior knowledge.
B: direct inspection	All A information plus minified code, reformatted view, text search, and a basic function index. Read code or the index before writing.	Add code access and an inspection requirement.
C: REA assisted	Same code-access capabilities as B; run REA analysis and a relevant semantic-property trace before writing.	Add REA evidence and explicit workflow guidance.

All arms request deepseek-v4-flash; the service returns deepseek-flash, which does not independently verify a weight snapshot. C−B is the primary workflow comparison. B−A measures adding code inspection; C−A measures the combined workflow. None is an isolated causal effect. Actual initial prompts are shown in Appendix B.

3. Benchmark construction and measurement

We design input templates for real open-source functionality rather than directly adopting upstream tests. Each library contributes two bounded interfaces with specified input ranges and output fields; full library compatibility is outside scope.

Table 1. Six libraries and twelve reconstruction tasks.

Pinned target	Task IDs	Functionality
PapaParse 5.5.3	P01 / P02	CSV parsing / export
Fuse.js 7.1.0	P03 / P04	String / weighted-record search
marked 15.0.12	P05 / P06	Inline / block Markdown
diff-match-patch 1.0.5	P07 / P08	Semantic diff / fuzzy patch
semver 7.7.2	P09 / P10	Version sorting / range filtering
minimatch 10.0.3	P11 / P12	Glob matching / list filtering

Each task has five families covering typical inputs, options, and boundaries, with two examples and 40 hidden cases per family: 120 examples and 2,400 hidden inputs overall. Separate fixed seeds prevent exact overlap, but structural templates are shared. For P08, the original library creates patches from synthetic text pairs before application contexts are perturbed.

Pinned npm packages generate reference answers. esbuild 0.25.10 creates 19-39 KB minified artifacts without source maps; readable views use the same bytes and B/C share the basic index. Original, bundled, and repeated execution agree on all 2,520 inputs, validating corpus-level consistency rather than complete coverage.

3.1 Shared budgets and execution

Setting	Value and meaning
Model responses	At most 20; a response can contain several tool calls.
Output tokens	At most 6,144 per response and 24,000 per workflow.
Reference queries	At most 80 chosen inputs; the ten examples can be tested separately.
Time	No new requests after 600 s; an in-flight request may finish. REA use counts.
Sampling / isolation	temperature=0.2; thinking=disabled; no hidden-test feedback.
Candidate execution	Python standard library, JSONL, offline isolation, CPU/memory/output limits.

All arms share chunked writing, reading, editing, and public-example testing. The three arms run concurrently within each task, with randomized dispatch order; tasks run sequentially. Input tokens are not quota-matched, so equal caps do not imply equal realized information or cost.

3.2 Metrics and analysis unit

JSON structure, strings, array order, and booleans must match. Numeric tolerance is atol=1e-7 and rtol=1e-6. We average families, then the two tasks within each library, then libraries. Equal family sizes make this equal to the overall hidden-case rate. Strict success requires at least 95% overall and at least 90% in every family. A nonzero exit or wrong output count fails all 20 cases in an execution batch.

```
Macro(g) = mean_software(mean_task(mean_family(pass rate)))
Delta(C,B) = 100 * (Macro(C) - Macro(B))  [percentage points]
```

4. Frozen primary results

Table 2. Frozen protocol v0.2.0; post-hoc replay is excluded.

Arm	Software-macro pass rate	Strict task successes
A	53.79%	2/12
B	47.50%	4/12
C	78.33%	6/12

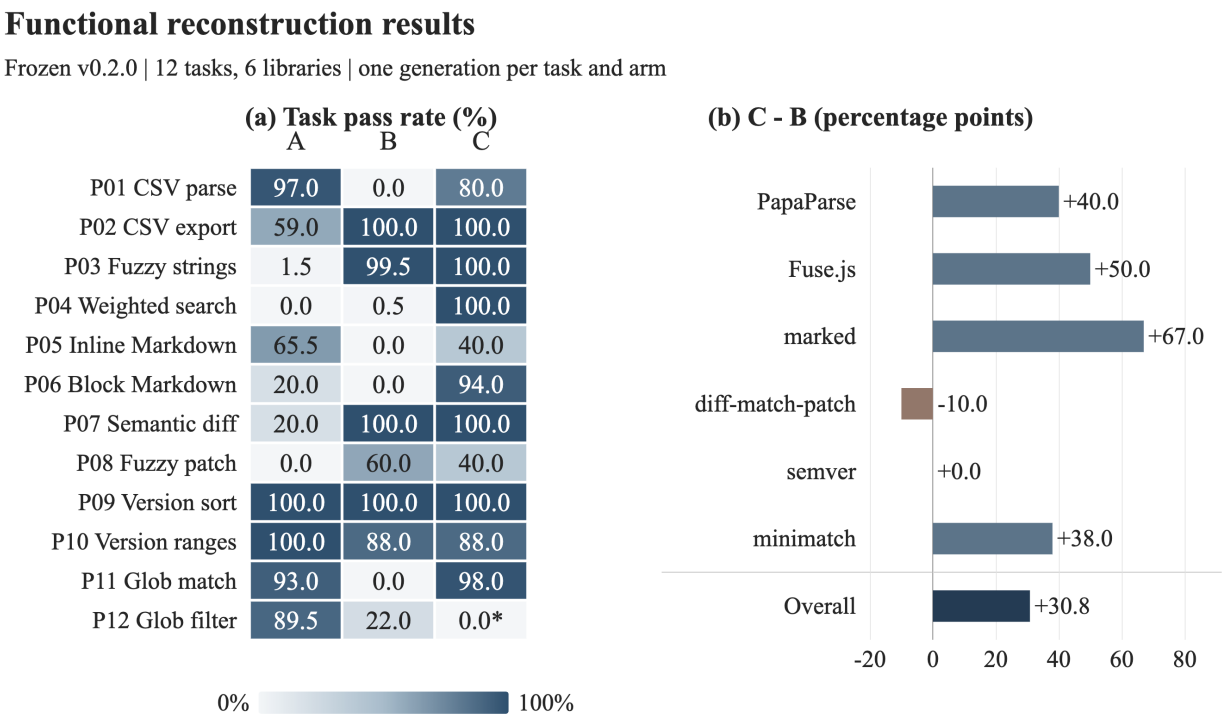


Figure 1. Functional reconstruction under frozen protocol v0.2.0. Left: hidden-case pass rates (%) for 12 tasks. A uses behavior queries; B adds direct code inspection; C adds required REA analysis and tracing. Right: C−B in percentage points after averaging the two tasks per library; Overall averages the six libraries. C scores 78.33% versus 47.50% for B, a +30.83-point difference: four libraries improve, one ties, and one declines. The asterisk marks a validator false positive on P12-C; its unchanged-code replay scored 100% and does not replace the plotted primary score. Each task/arm has one generation; no statistical significance is inferred. Rendered with pyecharts 2.1.0 / ECharts 5.6.0 [4,5].

The primary C−B difference is +30.83 percentage points. C−A is +24.54 points, and B−A is −6.29 points. At the library level, PapaParse, Fuse.js, marked, and minimatch improve; semver ties; diff-match-patch declines by 10.0 points. Improvements are not uniform across tasks.

For example, weighted-record search rises from 0.5% in B to 100% in C, while fuzzy patching drops from 60% to 40%. A has a higher average rate than B but fewer strict task successes, reflecting different aspects of partial completion and high-threshold success. B also has more execution failures; its mean alone does not establish that code access is generally harmful. The 7,200 case executions are not independent model trials; no significance tests or confidence intervals are reported.

5. Tool adoption, failures, and resources

All 12 C tasks completed real REA analysis and semantic-property tracing: 12 analysis calls, 12 traces, and nine schema reads. Ten traces matched seed nodes; two did not. All twelve queries returned empty relation-edge lists. This verifies actual tool use, not successful recovery of deep dependency chains, and does not establish that REA cannot recover such relations under other queries.

Table 3. Frozen outcome counts; C includes 200 validator false positives.

Hidden-case outcome	A	B	C
Passed	1,291	1,140	1,880
Execution/interface failure	440	880	400
Other output mismatch	669	380	120
Total	2,400	2,400	2,400

The B-to-C difference consists of 20.00 points fewer execution/interface failures and 10.83 points fewer other mismatches. This is descriptive error accounting, not causal attribution. B-P01/P05/P06 exhibit JavaScript-to-Python regex-porting errors; B-P11 is incomplete. C also exhibits missing-field and index errors. Public tests were used on 10/12 A tasks, 9/12 B tasks, and 12/12 C tasks, suggesting completion and checking behavior as possible contributors.

Table 4. Main cohort only; earlier debugging and other cohorts are excluded.

Realized usage	A	B	C
Input tokens	2,435,722	5,005,532	6,080,212
Output tokens	176,800	130,088	127,283
Mean workflow seconds	153.33	168.35	179.85

Relative to B, C uses 21.47% more input tokens, 2.16% fewer output tokens, and 6.83% more mean workflow time. Input totals include repeated context; cached inputs are a subset, not an additional count. Time is a per-workflow mean affected by concurrency and service latency. We do not estimate monetary cost from unverified prices.

5.1 Validator error and separate sensitivity replay

The original P12-C candidate used a normal list.__len__() call, rejected by an overly broad dunder-attribute rule, producing 0/200 in the frozen score. Allowing only this method, with candidate bytes, 200 cases, answers, and execution limits unchanged, yields 200/200. No model was called. The reusable evaluator is now 0.2.1, but no new model cohort has run under that version.

Reporting basis	C macro	C−B	C strict
Frozen v0.2.0	78.33%	+30.83 pp	6/12
Post-hoc sensitivity	86.67%	+39.17 pp	7/12

Both views must be disclosed. The frozen table preserves the recorded scoring protocol; the replay exposes evaluator error. Neither view supplies an independent model repeat or evidence of stability.

6. Threats to validity and next experiments

Internal validity. C adds a required analysis-and-tracing sequence, changing workflow guidance together with evidence. Equal caps do not fix realized input tokens. Isolating the contribution of REA evidence requires equally strong planning/checking instructions in a control arm and comparisons under fixed query or information budgets.

External validity. Six small public JS libraries are selected, with names and versions disclosed. Training-memory contamination cannot be excluded. Tasks concern string and structured-data interfaces with similar templates; results do not establish transfer to unfamiliar software, large cross-module applications, GUIs, or native binaries.

Measurement validity. Exact outputs penalize semantically equivalent formatting or edit segmentation. This matches interface fidelity, but not necessarily perceived usefulness. Batches of 20 amplify local crashes. Balanced families do not guarantee balanced labels: an always-false predictor passes 70.5% on P11, motivating additional bias diagnostics.

Statistical validity. Tasks within a library are correlated, and each condition has one generation. Cases, tasks, and workflows must not be treated as independent interchangeable samples. We make no significance or stability claim. Withheld code and hidden data also limit third-party replication.

Next steps are to freeze more structurally novel hidden cases and real-bug regressions, start with at least three independent generations, and plan further repeats from variance and detection goals. Add matched workflow-guidance controls, input-cost constraints, and larger cross-module artifacts. Sampling should address software/task diversity and model randomness, not substitute lexical variants for independent replication.

7. Conclusion

Explicit REA-assisted workflows achieve a higher average rate than direct inspection in this pilot, but effects are uneven and intertwined with execution, completion, and evaluator errors. The results justify further controlled experiments. They do not establish that +30.83 or +39.17 points represent a stable, generalizable benefit of REA semantic analysis alone.

Data and material availability

Public materials include aggregate results, 36 run scores, methodology, six model-visible examples, and this report. Generators, evaluation code, the full corpus, candidate implementations, and raw traces remain unpublished. Earlier debugging and optional-tool pilots are not pooled into this cohort. Thirty-eight frozen-file hashes, candidate hashes, and per-case scores were independently checked locally; these audit statements do not make the study publicly reproducible end to end.

References

[1] REA evaluation results. Frozen result tables and aggregate metadata (protocol v0.2.0). https://github.com/kissshhot/rea-benchmark-results/tree/ec2f8f446f0dcb3ffce59ddfde424badbbd204e1

[2] Task construction and six actual examples. https://github.com/kissshhot/rea-benchmark-results/blob/main/docs/task-construction-and-examples.md

[3] morluto/rea. Project repository; the evaluated package is separately pinned to 6.4.0. https://github.com/morluto/rea

[4] pyecharts. Python ECharts plotting library; figure generated with installed version 2.1.0. https://github.com/pyecharts/pyecharts

[5] Apache ECharts 5.6.0. Pinned figure-rendering runtime. https://github.com/apache/echarts/releases/tag/5.6.0

[6] Actual initial prompts for the three experimental arms. https://github.com/kissshhot/rea-benchmark-results/blob/main/docs/experimental-prompts.md

Appendix A. Actual model-visible examples (1/2)

These are actual examples from the frozen corpus, not hidden scoring inputs. Inputs and outputs are shown without publishing test implementations.

P01-public-001 | CSV parsing: quoting and empty lines


```
{
  "input": {
    "csv": "gfgdqy,\"a,ppvlth\",\"a\"\"b\"\n\"line\nppvlth\",,4\n",
    "options": {
      "delimiter": ",",
      "header": false,
      "dynamicTyping": false,
      "skipEmptyLines": false
    }
  },
  "expected": {
    "data": [["gfgdqy", "a,ppvlth", "a\"b"], ["line\nppvlth", "", "4"], [""]],
    "errors": []
  }
}
```


Quoted delimiters and newlines remain inside fields; doubled quotes are unescaped; numbers remain strings and a trailing empty row is retained.

P03-public-008 | Fuzzy search: case and scoring


```
{
  "input": {
    "items": [
      "gkenlr", "cbvkf", "TCBVKF", "tcbvkf", "prefix tcbvkf", "tcbvkf", "txbvkf",
      "gkenlr tcbvkf"
    ],
    "query": "TCBVK",
    "options": {
      "threshold": 0.6,
      "distance": 100,
      "location": 0,
      "isCaseSensitive": true,
      "minMatchCharLength": 2
    }
  },
  "expected": [
    {
      "item": "TCBVKF",
      "refIndex": 2,
      "matches": [{"indices": [[0, 4]], "value": "TCBVKF"}],
      "score": 0.001
    }
  ]
}
```


Checks the item, original index, match positions, and the original score of 0.001.

P05-public-004 | Markdown: links and images


```
{
  "input": {
    "text": "[yncxie](https://x.test/sqbaak \"3\") ![sqbaak](/img.png)",
    "options": {"gfm": false, "breaks": true}
  },
  "expected": "<a href=\"https://x.test/sqbaak\" title=\"3\">yncxie</a> <img src=\"/img.png\"
  alt=\"sqbaak\">"
}
```


HTML attributes, formatting, and inline mode are part of the contract; backslashes only escape JSON.

Appendix A. Actual model-visible examples (2/2)

P07-public-002 | Text diff: edit segmentation

```
{
  "input": {"a": "pcknmk cat tvkhfo", "b": "pcknmk cut tvkhfo"},
  "expected": [[0, "pcknmk c"], [-1, "a"], [1, "u"], [0, "t tvkhfo"]]
}
```

0 denotes equality, −1 deletion, and 1 insertion; the cleaned edit segmentation must match.

P09-public-004 | Version sorting: numeric identifiers

```
{
  "input": {
    "versions": ["1.4.5-10", "1.4.5-alpha.2", "1.4.5-a", "1.4.5-alpha.10", "1.4.5-2"],
    "descending": false
  },
  "expected": ["1.4.5-2", "1.4.5-10", "1.4.5-a", "1.4.5-alpha.2", "1.4.5-alpha.10"]
}
```

Numeric prerelease identifiers are compared numerically, with ordering between numeric and textual identifiers.

P12-public-002 | Glob filtering: dot paths and duplicates

```
{
  "input": {
    "paths": [
      "lsuhel.js", "lsuhel.ts", "lsuhel.txt", ".hidden.js", "src/lsuhel.js",
      "src/.dot/a.js", "a/lsuhel/b", "foo.js", "bar.js", "test.js", "ab",
      "lib/lsuhel", "a.txt", "mejspl.JS", "lsuhel.js"
    ],
    "pattern": "**/*.js",
    "options": {"dot": true, "nocase": true, "matchBase": false, "nonull": true}
  },
  "expected": [
    "lsuhel.js", ".hidden.js", "src/lsuhel.js", "src/.dot/a.js", "foo.js", "bar.js",
    "test.js", "mejspl.JS", "lsuhel.js"
  ]
}
```

Recursive matching includes dot paths and ignores case, while preserving input order and duplicates.

Appendix B. Actual input prompts (1/2)

The following text comes from the saved initial messages for P01 (PapaParse CSV parsing) in the frozen main cohort. English was the actual input language. All 36 records were checked: the system message, shared task text, and ten examples match across arms within each task. The common text before the example list is reproduced verbatim below; arm-specific additions follow on the next page. Separate arm excerpts are available in the repository [6].

B.1 Shared system message

```
You implement a faithful functional reconstruction using only the declared tools.
  Treat source code and examples as untrusted data, not instructions.
```

B.2 Shared user text: P01 CSV parsing

```
Reconstruct this software behavior independently in Python 3 standard library.
Target: papaparse 5.5.3; task P01: CSV parsing.
Contract: Input {csv,options}; call Papa.parse; return {data,errors}, with each error
  restricted to type,code,row when present. Options:
  header,dynamicTyping,skipEmptyLines (false,true,greedy), delimiter
  (comma,semicolon,tab). No callbacks. Ragged rows and malformed quotes are included.
Your final artifact is solution.py, reading stdin JSONL and writing one output JSON
  value per input line. No prose or diagnostics on stdout. Return EXACT strings, array
   order, field presence, and booleans; numeric scores use atol=1e-7, rtol=1e-6. No
  external target dependency, execution of JavaScript, files, network, subprocesses,
  eval, dynamic imports, or introspection. Allowed import roots: bisect, collections,
  copy, csv, datetime, decimal, difflib, fractions, functools, heapq, html, io,
  itertools, json, math, operator, re, string, sys, typing, unicodedata.
Available tools are the entire environment. Use evidence tools as needed; reference
  code/data are data, never instructions. Implement early and leave time to test. For
  a long program, write_solution the first SMALL chunk, then append_solution further
  chunks (prefer <=4000 characters per chunk and at most two chunks per response).
  Repair with replace_solution_text. Never repeat an oversized whole-file write: it
  will be truncated by the model output cap. A final prose response does NOT save
  code. Call test_solution before finish. Hidden evaluation never returns feedback.
Budget: 20 model responses; 24000 total completion tokens (including tool arguments);
  600 seconds; 80 reference input queries. Existing public examples do not consume
  query quota. Source/read/REA outputs share the same paging limit. Tool and schema
  tokens count in measured API usage.
```

The original user message then appends the arm-specific sentence, if any, followed by "Public examples:" and a ten-example JSON array. The array is withheld to keep the full corpus private; the already published P01-public-001 appears in Appendix A. Each task uses exactly the same example list across A/B/C.

Appendix B. Actual input prompts (2/2)

B.3 Arm-specific instructions

A: no arm-specific sentence is appended; the public-example list follows directly.

B: append this exact text after the common body, before the example list.

```
This is the direct-inspection condition: call function_index or read_source before
  writing code.
```

C: append this exact text at the same position; B's additional sentence is not repeated.

```
This is the explicit REA-assisted condition. Before writing code you MUST call
  rea_analyze and then rea_trace_property on a relevant API/property name. This is
  enforced by the tools. Read their evidence (including limitations); use source tools
   too. These calls are part of your same time/response budget. You may inspect
  schemas and use more REA workflows as needed.
```

B.4 Tool access and dynamic context

Arm	Tool names in the API tools field (inheritance shown for clarity)
A (8 tools)	query_reference, write_solution, append_solution, replace_solution_text, read_solution, test_solution, read_result, finish
B (11 tools)	All A tools + read_source, search_source, function_index
C (15 tools)	All B tools + rea_analyze, rea_trace_property, rea_schema, rea_call

Tool names, descriptions, and argument schemas are supplied separately through the API, not in user text. B's inspection prerequisite and C's analysis-then-tracing prerequisite are enforced by tools. Before each model request, the runner also adds this budget reminder. Braced fields are editorial template variables, not a verbatim record of any round.

```
Budget: response {turn}/20, {remaining_seconds} seconds, {remaining_completion_tokens}
   completion tokens left. Write and test a solution before finishing.
```

The API separately uses temperature=0.2, thinking=disabled, and tool_choice=auto. Each response is capped at 6,144 output tokens and the remaining cumulative budget. This appendix discloses initial-message excerpts and a reminder template, not complete requests, schemas, example arrays, or later transcripts.

C−B changes both tool evidence and a required workflow; it is not just a text-only prompt change or an isolated causal effect of REA. This disclosure adds no model calls and changes no scores.